



MATH-454: PARALLEL AND HIGH-PERFORMANCE COMPUTING

Series 5

Author:

Mattia BARBIERE (387974)

Professor:

Pablo ANTOLIN

Spring Semester - 2025

1 Assessing inefficiencies: Profiling

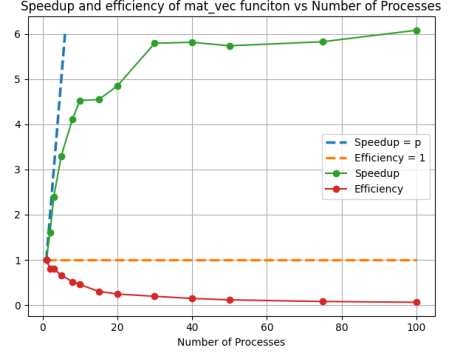
The first step to any parallelization is understanding the bottleneck in the code. After taking some time to understand the code I performed a sequential profiling with `perf` on the unaltered code. Please note that throughout this report the code was compiled without any optimizer so that results are more evident.

From Figure 1a it is clear that the main bottleneck that we can control is the `mat_vec` function that performs matrix vector multiplication on the sparse matrix. By simple inspection the `CGSolverSparse::solve` function, that implements the conjugate method, is inherently sequential and thus cannot be parallelized.

Samples: 121K of event 'cycles:u', Event count (approx.): 103253420871,

	Children	Self	Command	Symbol
+	83.90%	0.00%	cgsolver	[.] main
+	83.87%	0.00%	cgsolver	[.] CGSolverSparse::solve
+	82.86%	10.50%	cgsolver	[.] MatrixC00::mat_vec
+	29.04%	2.61%	cgsolver	[.] std::ios_base::setf
+	28.48%	2.09%	cgsolver	[.] std::operator
+	15.09%	1.63%	cgsolver	[.] std::operator&=
+	9.13%	0.54%	cgsolver	[.] std::scientific
+	9.09%	0.30%	cgsolver	[.] MatrixC00::n
+	2.71%	2.71%	cgsolver	[.] __cxa_pure_virtual@plt
+	2.57%	0.00%	cgsolver	[.] _dl_relocate_static_pie

(a) Sequential profiling with `perf` on the code before parallelization.



(b) Average speedup and efficiency of the `mat_vec` function when applied to `lap2D_5pt.n1000.mtx` as a function of the processor count.

Figure 1

2 Dividing the work: MPI parallelization

After having found the candidate for parallelization the next step was the implementation using MPI. The crux of the `mat_vec` function is the `for` loop that performs the multiplication and addition of the matrix and vector. The strategy I invoked to parallelize this function was to evenly split the non-zero indices of the matrix among the given processors. The processors would then compute the additions and multiplication only on their portion of the matrix and have zero everywhere else. Lastly I performed a `MPI_Allreduce` with the `MPI_SUM` operation to combine all the results. Next I timed how long the `mat_vec` function took to compute as I varied the number of processors. The speedup and efficiency of the `mat_vec` function are presented in Figure 1b.

We clearly see that there is an initial speedup when we go from 1 processor to 25, however after this there is a evident plateau where the additional processors have no effect on the speed up. For the same reason the efficiency starts very close to one but eventually decays to 0 as additional processors are of little help. This is the first indication that the code might not scale that well and in fact the same pattern emerges from weak and strong scaling.

3 The bigger picture: Strong and weak scaling

The parallelization of the `mat_vec` function is only one part of the code. We need to look at the code in its entirety. To estimate the parallelizable code in Amdahl's law I measured the proportion of time that the `mat_vec` function was running compared to the total time of the whole code and I got $\alpha \approx 0.75$. Hence, according to Amdahl's law, the ideal speedup should be $S(p) = \frac{1}{0.25 + \frac{0.75}{p}}$ and so should tend towards 4 as the number of processors p goes towards infinity.

Unfortunately, this is far from what happens in practice. In Figure 2 we see that the empirical speedup starts close to the ideal value but then deviates very early with only a slight gain for few processors. In fact as the number of processes increases we actually get a slower performance as the speedup drops below 1. This is likely attributed to the memory and communication bottlenecks. With more processors there is more data to move around and moreover having more processors does not improve the speed of memory read and write. Next to perform weak scaling and apply Gustafson's law I needed to construct matrices of the correct size so that the work per processors is constant. I started from the `lap2D_5pt.n2000.mtx` and extracted the top left portion of various sizes. In the end I ended up with the amount of non-zero entries per process at about $k = 119780$. I then timed the code with p processors keeping non-zero entries per processor constant and compared it to the performance of a single processor on the same matrix. The ratio gave me the values of $\alpha(N)$ show in Table 1. Gustafson's law then tells us that the speedup should be $1 + (p - 1)\alpha(N)$ which is

plotted in red on Figure 3. Similarly to before both the speedup and the efficiency start close to the ideal but also deviate quickly. We see the same pattern that there is an effective speed up in the code only for low values of processor counts. As soon as the number of processor goes above 20 the extra processors only add memory and communication complexity without any added benefit in speed. The reason behind this is again that the problem becomes memory bound and adding additional processors will not help this, and actually ends up hurting the performance because of the additional complexity in communication.

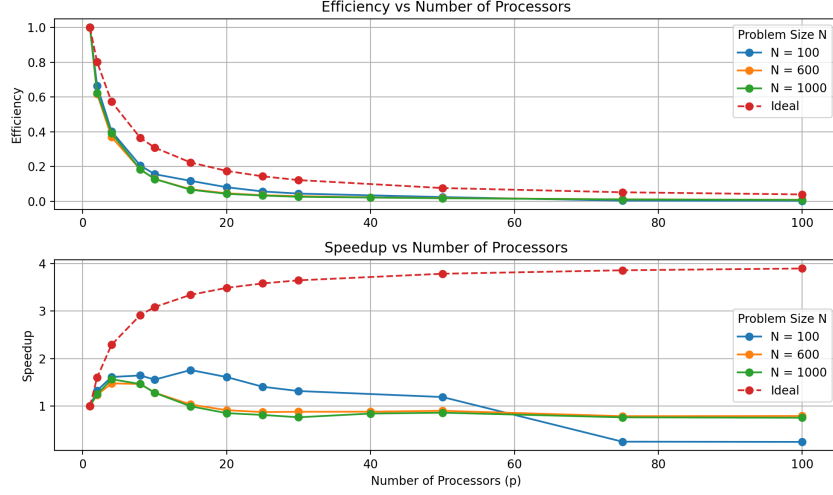


Figure 2: Efficiency and speedup after strong scaling and the ideal efficiency and speedup according to Amdahl’s law.

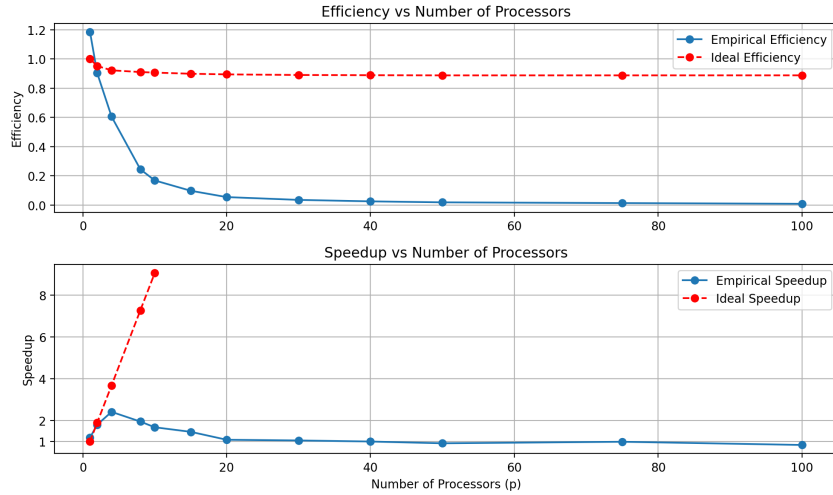


Figure 3: Efficiency and speedup after weak scaling and the ideal efficiency and speedup according to Gustafson’s law.

N	k	$2k$	$4k$	$8k$	$10k$	$15k$	$20k$	$30k$	$40k$	$50k$	$75k$	$100k$
$\alpha(N)\%$	88.69	90.39	89.61	89.75	89.61	89.14	88.87	88.63	88.58	88.53	88.61	88.67

Table 1: Estimated value of the parallelization proportion of the code for weak scaling. The amount of non-zero entries per processor was taken to be $k = 119780$.

4 Conclusion and further considerations

This report focuses on parallelizing the conjugate gradient method for solving linear systems, with an emphasis on the `mat_vec` function. I evaluated its performance under both strong and weak scaling. Although the main effort was on parallelizing `mat_vec`, further optimizations are possible. As the code is now memory-bound, allocating only the relevant matrix segments to each processor could enhance performance further.